Credit Application - Documentation

The purpose of this document is to define the latest Water Desk Credit App Aspire API call JSON documentation - everything for the booking process. POST, PUTS, GETS, and even DELETES (if applicable), that may be a part of the process.

Steps

Full step-by-step walkthrough

New customer flow

1. Send Credit Application is clicked.
2. TeamDesk logs start time and gets dealer info.
3. **Step 1** creates customer in Aspire using New Customer Account(Step 1) with `POST /Account`.
4. TeamDesk moves to Step 2 to create location.
5. Step 3 assigns/sets primary location.
6. **Step 4** creates the contract in Aspire with Send Contract to Aspire(New Customer), Step 4 using `POST /Contract`, setting status to **Dealer Submitted**.
7. TeamDesk moves to Step 5 and later steps for contract info, assets, payment info, related accounts, billing-location variants, etc.
8. Status is checked later manually or by periodic triggers.
9. Aspire returns approval/decline; TeamDesk stores it in CreditDecision / ContractStatus.

Existing customer flow

Uses Send Contract instead of creating a new customer first.

---

1. New Customer Account(Step 1)

Description: We send the customerâ€™s business information to Aspire so the customer exists in their system.

Why this step exists: Aspire cannot process the credit application until the customer account has been created.

What TeamDesk does: TeamDesk sends the companyâ€™s legal name, EIN, contact information, and customer record ID to Aspire and creates the customer as a business account.

Result: A customer account is created in Aspire and is ready to be used for the rest of the credit application process.

Syntax	Description
Method	POST
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/Account`
Headers	`Content-Type: application/json`
Body	see JSON below

```
{
"AccountType": "Business",
"Name": "<%[Company Full Legal Name]%>",
"CompanyType":{"Code":"C","Description":"Corporation"},
"Status":"Active",
"LegalName": "<%[Company Full Legal Name]%>",
"Email":"<%[New Email]%>",
"DoingBusinessAs":"<%[DBA:]%>",
"EstablishedDate": "<%ToText([Business Start Date])%>",
"FederalIdentificationNumber": "<%[EIN]%>",
"IsPrimary":true,
"PhoneNumbers": [{
"Number": "<%[AS1-Phone]%>",
"Extension":null,
"PhoneNumberType": "Main",
"IsPrimary": true
}],
"StateOfIncorporation":null,
"Contacts": [{"Value": "<%[Customer Account#]%>","Type": "Record"}],
"RecordId" : {"Type":"Record", "Value":"<%[Customer Account#]%>"},
"Roles":[{
"RoleType": "CUST",
"Description": "Customer"
}]
}
```

2. Create Location(New Customer) Step 2

Description: We send the customerâ€™s main address to Aspire so the customer has a location on file.

Why this step exists: Aspire needs an address/location for the customer before the contract can be completed correctly.

What TeamDesk does: TeamDesk sends the customerâ€™s main address, city, state, country, postal code, contact name, email, phone number, and tax-exempt status to Aspire as the primary location.

Result: A primary customer location is created in Aspire and linked to the customer account.

Syntax	Description
Method	POST
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/location`
Headers	`Content-Type: application/json`
Body	see JSON below

```
{
  "EntityRecordId":"<%[Customer Account#]%>",
  "Code":"Primary Location",
  "RecordId": "<%[Customer Account#]%>",
  "AccountType": "Business",
  "IsPrimary": true,
  "IsTaxExempt":"<%If([Is Tax Exempt]="Yes","True","false")%>",
  "Location":"Location",
  "Description":"<%[New City] &"Primary Location"%>",
  "Address1": "<%[New Main Address]%>",
  "Address2": "<%[New Main Address#2]%>",
  "City": "<%[New City]%>",
  "State": "<%[New State]%>",
  "Country": <%=If([Country] = "Canada", "CAN", IsNull([Country]), "USA", [Country])%>,
  "Attention": "<%[AS1-First Name]&" "&[AS1-Last Name]%>",
  "Email": "<%[AS1-Email]&"; "&[AS2-Email]%>",
  "PhoneNumbers": [{
"Number": "<%[AS1-Phone]%>",
"Extension":null,
"PhoneNumberType": "Main",
```

```
"IsPrimary": true
}],
"Name":"Primary Location",
"AddNew":true,
"IsActive":true,
"PostalCode": <%=If([Country] = "USA", [New Zip/Postal Code], [Country] = "Canada", [New Postal Code], null)%>,
"LocationRecordId": {
    "Type": "Record",
    "Value": "<%[Customer Account#]%>"
  }
}
```

3. Set Primary Location(New Customer) Step 3

Description: We tell Aspire which location should be treated as the customerâ€™s main location.

Why this step exists: Even after the address is created, the customer account still needs to be pointed to that location as its primary location.

What TeamDesk does: TeamDesk updates the customer account in Aspire so the newly created location becomes the accountâ€™s official primary location.

Result: The customer account now has the correct primary location assigned in Aspire.

Syntax	Description
Method	PUT
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/Account`
Headers	`Content-Type: application/json`
Body	see JSON below

```
{
  "RecordId": {"Type":"Record", "Value":"<%[Customer Account#]%>"},
  "LocationRecordId": {
    "Type": "Record",
    "Value": "<%[Customer Account#]%>"
  }
}
```

4. Send Contract to Aspire(New Customer) Step 4

Description: We create the credit application in Aspire as a contract and submit it for review.

Why this step exists: This is the actual submission of the credit application to Aspire.

What TeamDesk does: TeamDesk sends the contract number, customer ID, bill-to location, tax location, dates, term, billing setup, and related dealer/admin information to Aspire. The contract is created with status Dealer Submitted.

Result: The credit application exists in Aspire as a contract and is officially submitted for processing.

Syntax	Description
Method	POST
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/Contract`
Headers	`Content-Type: application/json`
Body	see JSON below

```
{
  "RecordId": {"Type": "Record","value": "<%[Contract#]%>"},
  "FinanceCompanyRecordId": {"Type": "Record", "value": <%= If([Country] = 'Canada', "10002", "10001") %> },
  "CustomerRecordId": {"Type": "Record","value": "<%If([Mavis Account Override]=True,[Mavis Account Override ID],[Account No#])%>"},
  "ContractStatus": {
      "ContractRecordId":  {"Type": "Record","value": "<%[Contract#]%>"},
      "Status": "Dealer Submitted"
    },
  "BillToLocationRecordId": {"Type": "Record","value": "<%If([Mavis Account Override]=True,[Mavis Account Override ID],[Customer Account#])%>"},
  "TaxLocationRecordId":  {"Type": "Record","value": "<%If([Mavis Account Override]=True,[Mavis Account Override ID],[Customer Account#])%>"},
    "SignDate": "<%ToText([Close Date])%>",
    "StartDate": "<%ToText([Close Date])%>",
    "Loan": null,
    "Term": "<%[Term(New)]%>",
    "TransactionCode": {
      "InvoiceDescription": "Rent",
      "Code": "RENT",
      "Description": "Rent"
    },
    "AccountDistributionCode": {
      "Code": null,
      "Description": null
    },
    "InterestCalculation": 70,
    "FinanceProgram": {
      "Code": "NON-RECOURSE",
      "Description": "Non-Recourse"
    },
    "FinanceProduct": {
      "Code": "Rental Contract",
      "Description": "Rental Contract"
    },
    "ContractType": {
      "Code": "NON RECOURSE",
      "Description": "Non Recourse"
    },
    "ContractCategory": {
      "Code": null,
      "Description": null
    },
    "DelinquencyCode": {
      "Code": "15-15",
      "Description": "15 Days 15%"
    },
    "InvoiceLeadDays": {
      "Code": "<%If([Billing Freq(New)]="Monthly","20",
                  [Billing Freq(New)]="Quarterly","45","20")%>",
      "Description":"<%If([Billing Freq(New)]="Monthly","20 Days all Monthly Contracts",
                        [Billing Freq(New)]="Quarterly","45","20")%>"
    },
    "PenaltyCode": {
      "Code": null,
      "Description": null
    },
    "InvoiceCode": {
      "Code": "BTS",
      "Description": "BillTrust-Separate"
    },
    "CompoundingPeriod": "FollowPaymentFrequency",
    "ComputeMethod": "Normal",
```

```
    "YearLength": "YearLength360",
    "SuspendInvoicing": true,
    "Eft": null,
    "RelatedAccounts": [
      {
        "ContractId": {"Type": "Record","value": "<%[Contract#]%>"},
        "EntityId": {
          "Value": "<%[Administrator Id]%>",
          "Type": "Transaction"
        },
        "Role": {
          "RoleType": "OTHER4",
          "Description": "Dealer Administrator"
        },
        "IsPrimary": true,
        "Action": "Default",
        "RecordId": null
      },
      {
        "EntityName": "<%[Dealer Name]%>",
        "ContractId": {"Type": "Record","value": "<%[Contract#]%>"},
        "EntityId": {
          "Value": "<%[Dealer No]%>",
          "Type": "Transaction"
        },
        "Role": {
          "RoleType": "VEND",
          "Description": "Dealer"
        },
        "IsPrimary": true,
        "Action": "Default",
        "RecordId": null
      }
      ]
}
```

5. [Credit Application: ggg](https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/Contract/<%[Contract#]%>/record)

Description: We ask Aspire for the contract details so TeamDesk can store important information from the submitted application.

Why this step exists: Later steps depend on values that only Aspire can provide after the contract has been created, especially the transaction number.

What TeamDesk does: TeamDesk retrieves the contract record from Aspire and stores items such as contract status, credit decision, decision date, approval amount, and the Aspire transaction number (Trans#).

Result: TeamDesk now has the contract details it needs to continue the process and track the application.

a) Get Contract Info(New Customer Only)

Syntax	Description
Method	GET
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/Contract/<%[Contract#]%>/record`
Headers	`Content-Type: application/json`

b) Get Credit Status Aspire

Syntax	Description
Method	GET
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/contract/<%Trim(ToText(Format([Trans#])))%>/transaction`
Headers	`Content-Type: application/json`

6. Attach the Assets for Equipment(Primary)

Description: We send the main piece of equipment or unit to Aspire and attach it to the contract.

Why this step exists: Aspire needs to know what asset or equipment is part of the financing agreement.

What TeamDesk does: TeamDesk sends the asset description, asset code, serial number, quantity, funding amount, location, vendor/distributor, and contract transaction number to Aspire.

Result: The primary asset is created in Aspire and associated with the submitted contract.

Syntax	Description
Method	POST
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/asset`
Headers	`Content-Type: application/json`
Body	see JSON below

```
[
{
"Action": "6",
"Description": "<%[Description]%>",
"ContractAssetId" :"<%[Asset Code]%>",
"ContractIdentifier":"<%ToText(Format([Trans#]))%>",
    "IsPrimary":true, "IsNew": true, "SerialNumbers":["<%[Asset SerialNo]%>"],
    "Model":"<%[Description]%>",
    "IsSoftCost": false,
    "Locationid":"<%[AssetLoc_id]%>",
    "LocationEntityRecordId": {"Type": "Record","value": "<%If([Mavis Account Override]=True,[Mavis Account Override ID],[Assetloc_id])%>"},
    "BillToLocationRecordId": {"Type": "Record","value": "<%If([Use Customer Address]=false,[BilltoLoc],[Mavis Account Override]=True,[Mavis Account Override ID],[Assetloc_id])%>"},
    "OriginalCost": "<%ToText([Asset_Funding])%>",
    "VendorRecordId": {
            "Type": "Id",
            "Value": "<%[Distributor Account#]%>"},
    "Quantity": "<%Left(ToText([Asset Qty]),".")%>",
    "RecordId":"<%ToText([Asset Id])%>",
    "AssetType": { "Code": "<%[Asset Code]%>", "Description": "<%[Description]%>"
    }
}
]
```

7. Tie Assets for the Multi Select(Primary) _Post

Description: We add the asset into the Aspire contract transaction so Aspire treats it as a financed contract item.

Why this step exists: Creating the asset alone is not enough; the asset must also be attached to the contract transaction structure.

What TeamDesk does: TeamDesk posts the primary asset into the Aspire contract transaction using the transaction number returned from earlier contract steps.

Result: The asset becomes part of the contract transaction in Aspire and is ready for final location and payment handling.

Syntax	Description
Method	POST
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/contract/<%ToText(Format([Trans#]))%>/transaction/assets`
Headers	`Content-Type: application/json`
Body	see JSON below

```
[
{
"Condition": "New",
"IsPrimary":true,
"ContractAssetId": {"Type": "Record","value": "<%[Asset Id]%>"},
"IsSoftCost": false,
"ListPrice": "<%ToText([Actual Funding Amount])%>",
"RecordId": "<%[Asset Id]%>",
"UnitAmount": "<%ToText([Actual Funding Amount])%>",
"OriginalCost": "<%ToText([Actual Funding Amount])%>",
"CarryingCost": "<%ToText([Actual Funding Amount])%>",
"FairMarketValue" : "<%ToText([Actual Funding Amount])%>",
"InstallationDate":"<%ToText([Close Date])%>"
}
]
```

8. Send the Payment Information(Primary)

Description: We tell Aspire how the customer will pay for the contract.

Why this step exists: The contract is not financially complete until the payment schedule and amounts are defined.

What TeamDesk does: TeamDesk sends the term, contract start date, funding amount, billing frequency, number of payments, and calculated payment amount to Aspire. It also adjusts the payment start date and amount depending on whether billing is monthly, quarterly, semiannual, or annual.

Result: The contract in Aspire now has its payment schedule and financial terms set up.

Syntax	Description
Method	PUT
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/Contract`
Headers	`Content-Type: application/json`
Body	see JSON below

```
{
"FinanceCompanyRecordId": {"Type": "Record", "value": <%= If([Country] = 'Canada', 10002, 10001) %> },
"CustomerRecordID": {"value": "<%If([Mavis Account Override]=True,[Mavis Account Override ID],[Customer Account#])%>","type": "Record"},
"RecordId": {"value": "<%[Contract#]%>","type":"Record"},
"ContractStatus": {"Status":"Dealer Submitted"},
"Term": "<%[Term(New)]%>",
"StartDate": "<%ToText([Close Date])%>",
"OriginalCost": "<%ToText([Potential Funding])%>",
"Lease": {
"Payments": [
  {
  "StartDate": "<%If([Billing Freq(New)]="Quarterly",ToText(AdjustMonth([Close Date],3)), [Billing Freq(New)]="SemiAnnual",ToText(AdjustMonth([Close Date],6)),
                  [Billing Freq(New)]="Annual",ToText(AdjustMonth([Close Date],12)),ToText(AdjustMonth([Close Date],1)))%>",
  "Occurrences": "<%Left(ToText([No of Payments]),".")%>",
  "Frequency": "<%[Billing Freq(New)]%>",
  "Amount": "<%If([Billing Freq(New)]="Quarterly",ToText([Total_Monthly_Payment_Amount]*3), [Billing Freq(New)]="SemiAnnual",ToText([Total_Monthly_Payment_Amount]*6),
                [Billing Freq(New)]="Annual",ToText([Total_Monthly_Payment_Amount]*12),ToText([Total_Monthly_Payment_Amount]))%>",
  "PaymentType": "Standard"
      }

    ]
  }
}
```

9. Move Asset to Contract(Primary)

Description: We connect the asset to the correct contract location and billing location in Aspire.

Why this step exists: The asset needs to point to the right customer/service location and, if applicable, the right bill-to location.

What TeamDesk does: TeamDesk updates the asset inside Aspire so it uses the correct customer location and bill-to location for the contract.

Result: The asset is fully connected to the contractâ€™s final location and billing structure in Aspire.

Syntax	Description
Method	PUT
Url	`https://<%[Aspire Http Link]%>.leaseteam.net/LeaseTeam.Aspire.Api/1/contract/<%[CA-ContractID]%>/record/asset`
Headers	`Content-Type: application/json`
Body	see JSON below

```
[
{
"IsPrimary":true,
"RecordId": {
        "Value": "<%[Asset ID]%>",
        "Type": "Record"
      },
"LocationId":"<%If([Mavis Account Override]=True,[Mavis Account Override ID],[Customer Account#])%>",
"BillToLocationId":"<%If([Use Customer Address]=false,[BilltoLoc],[Mavis Account Override]=True,[Mavis Account Override ID],[Customer Account#])%>"
}
]
```

#. Credit Application:

Syntax	Description
Method	POST
Url	
Headers	`Content-Type: application/json`
Body	see JSON below

Record Change Triggers

The Credit Application process does not end when the user clicks **Send Credit Application**. After the initial booking flow, TeamDesk uses workflow triggers to keep the Aspire contract synchronized when status, billing, or term-related values change. These triggers support three important follow-up scenarios in the current database:

1. Status sync from Aspire back into TeamDesk
2. Recalculating and resending payment information after a term or billing change
3. Creating and applying a separate billing location after submission/approval

TeamDesk supports this trigger-driven automation pattern with record change and periodic triggers, plus linked workflow actions such as Call URL, Update Record, Create Record, and Delete Record

1. Update ContractStatus From Aspire

Purpose: This trigger keeps TeamDeskâ€™s contract status aligned with Aspire after the application has already been submitted.

When it runs:

- **Type:** Periodic
- **Schedule:** Daily at **3:00 AM**

Which records it checks:

This trigger processes Credit Application records where:

- ContractStatus is not `"Booked"`
- Date Created is within the last 90 days
- ContractStatus is different from Status From Contract Tab

What TeamDesk does:

The trigger runs:

- Get Credit Status Aspire(Step 5.2, slim)

This trigger is specifically used to ask Aspire for current status information and refresh TeamDesk fields when Aspire has newer contract status than what TeamDesk currently shows.

Why this matters: This is one of the main ways the database stays synchronized after the original booking flow finishes. It confirms that Aspire remains the source of truth for later contract status changes.

Result: If Aspireâ€™s contract status has changed, TeamDesk updates the local Credit Application record to reflect Aspireâ€™s latest contract/decision information.

2. Term Change(Equipment Asset Only)

Purpose: This trigger reruns rating, funding, and payment logic when the contract term or billing frequency changes on a submitted application that only has the main Equipment asset.

When it runs:

- **Type:** Record Change
- **Runs when record is:** Modified

Which records it checks:

This trigger only applies when:

- Credit Application Status is `"Submitted"`
- Description is `"Equipment"`

What fires trigger: The trigger runs when any of these values change:

- Billing Freq(New)
- Billing frequency
- Term(New)

What TeamDesk does: This trigger runs the following actions in sequence:

1. Remove the Rate Sheet Deletes the current rate information so it can be rebuilt.
2. Get the Rate Re-queries the Rate Sheet data for the updated term/frequency combination.
3. Update the Funding Information on the Asset Updates funding values on the asset based on the new rate.
4. Update the Rate Sheet Step(1) Updates the selected rate usage flags.
5. Asset TermChange Updates attached asset term-change values.
6. Update the Funding Information (Term Change) Recalculates asset funding for the new term.
7. Send the Payment Information(Primary), Step 8 Re-sends the payment stream to Aspire using the current term and billing frequency.
8. Term Change(Equipment Only) Navigates into the next related term-change step.

Why this matters: This trigger proves that Step 8 is not only part of the original booking flow. In this database, it is also the core payment update mechanism for post-submission term changes.

Result: When the term or billing frequency changes after submission, TeamDesk clears and rebuilds the rate/funding values and then pushes the updated payment stream back to Aspire.

3. Create Billing Location

Purpose: This trigger creates and applies a separate billing location in Aspire after submission or approval when the billing address is different from the customerâ€™s main address.

When it runs:

- **Type:** Record Change
- **Runs when record is:** Modified

Which records it checks: This trigger applies when:

- Bulk Load Units = `"No"`
- Use Customer Address = `false`
- and ContractStatus is one of:
  - `"Dealer Submitted"`
  - `"Approved by PWP (A)"`
  - `"Approved by PWP (B)"`
  - `"Approved by PWP (Recourse)"`

What fires trigger: The trigger runs when any of these values change:

- B-Street 1
- B-City
- Use Customer Address

What TeamDesk does: This trigger runs the following actions:

1. Create Location(Bill To Location) Creates a separate bill-to location in Aspire.
2. Update the Contract for Bill to Updates the Aspire contract so it uses the new bill-to location.

3. Update Payment Stream(BillToChange) Re-sends the payment stream while preserving contract status.
4. Update Units Billto (Attach Units) Updates attached-unit records so their bill-to references match the new billing location.

Why this matters: This is the clearest confirmed bridge between the Credit Application table and attached-unit logic. Even though I could not fully inspect the separate Attach Units table setup pages, this trigger confirms that attached units are part of the bill-to synchronization process.

Result: When a separate billing address is used after submission/approval, TeamDesk creates the bill-to location in Aspire, updates the contract to use it, refreshes the payment stream, and updates attached-unit bill-to values.

Attaching Units

The Credit Application process can include additional unit records beyond the main Equipment asset. In this database, attached-unit handling is confirmed by the dedicated Attach Units table, its relation back to Credit Application, and the attached-unit bill-to synchronization workflow.

What this means in the current database

Attached units are stored as child records in Attach Units through the reference Credit Application. The table also carries lookup fields from the parent Credit Application such as:

- CA-ContractID
- Transaction#
- BilltoLoc
- Use Customer Address
- ContractStatus

This confirms that attached units are part of the same contract/application context, not a separate disconnected process

Why this step exists: The primary Equipment asset is only one part of the overall financed package. Additional units need to remain tied to the same Aspire contract, location, billing, and term-related information so the contract stays internally consistent.

What TeamDesk does: In the current setup, TeamDesk uses child-table automation for attached units in at least two confirmed ways:

1. **Billing-location synchronization from Credit Application**
    - The parent Create Billing Location and Create Billing Location (Bulk Units) triggers both run Update Units Billto (Attach Units).
    - This confirms the parent Credit Application pushes bill-to updates into attached-unit records.
2. **Attached-unit Phase 3 contract move**
    - On the Attach Units table, the active trigger Update Units Billto runs for records where Phase contains "3" and Billto Change Date differs from Date Modified.
    - That trigger runs Move Asset to Contract, a Call URL action whose note says it attaches the location to the asset.

Result: Attached units are confirmed to be child records of the Credit Application and are actively synchronized when billing/location changes occur. In this database, TeamDesk updates attached-unit records after bill-to changes and then pushes the revised asset-location relationship back to Aspire for Phase 3 records.

Adding Documents

Document submission is handled by separate buttons depending on whether the Credit Application already has a Water Desk **DBKEY#**. In this database, document handling is clearly split into two paths: pull from Opportunity when a DB key exists, or navigate the user to manual document entry when no DB key exists. TeamDesk supports this kind of custom-button and Navigate-based workflow structure.

1. Attach Document

Description: This button is used when the Credit Application already has a DBKEY# and the packet/document can be retrieved from Opportunity.

When it is available: The button is shown when:

- Total Number of Units = Total Units Submitted
- Document Status is blank
- DBKEY# is populated
- ContractStatus contains "Approved"
- user Goods&Service Access is false
- override rules allow submission

What TeamDesk does: This button runs:

1. Get Opportunity Information(Data Only)
2. Get Opportunity Document
3. Update the Opportunity Flag
4. Remove the xml Doc

Why this step exists: If Opportunity already holds the document and the DB key links the records together, TeamDesk can fetch the document directly instead of asking the user to upload it manually.

Result: The document is pulled from Opportunity into the Credit Application process, and the related document-tracking fields are updated.

2. Add Your Document

Description: This button is used when there is **no DBKEY#** and the user must provide the document manually.

When it is available: The button is shown when:

- Total Number of Units = Total Units Submitted
- Document Status is blank
- DBKEY# is blank
- approval / override conditions are met

What TeamDesk does: This button runs Document Navigate, a Navigate action, which sends the user into the manual document-entry/upload process.

Why this step exists: When there is no linked Opportunity record via DB key, TeamDesk cannot fetch the file automatically, so the document must be added manually.

Result: The user is routed into the manual document-upload path and can add the required credit package directly.

Bulk Units

Bulk-unit processing is a variation of the Credit Application flow that flags the application as bulk and then uses a separate post-submission billing-location workflow.

1. Bulk Unit Application

Description: This button converts a submitted non-bulk application into the bulk-unit process.

When it is available: The button is shown when:

- Credit Application Status = "Submitted"
- Bulk Load Units is blank or "No"

What TeamDesk does: This button runs Update the Bulk-Load Field, an Update Record action.

Why this step exists: Bulk-unit applications use different downstream logic than standard credit applications, especially when billing changes must be propagated across many units.

Result: The Credit Application is flagged as a bulk-unit application and becomes eligible for the bulk-specific workflows.

2. Create Billing Location (Bulk Units)

Description: This trigger handles separate bill-to processing for applications marked as bulk.

When it runs:

- **Type:** Record Change
- **Runs when record is:** Modified

Which records it checks: This trigger applies when:

- Bulk Load Units = "Yes"
- Use Customer Address = false
- ContractStatus is one of:
  - "Documents Submitted"
  - "Approved by PWP (A)"
  - "Approved by PWP (B)"
  - "Approved by PWP (Recourse)"

What fires trigger: The trigger runs when any of these change:

- B-City
- B-Street 1
- Use Customer Address

What TeamDesk does: This trigger runs:

1. Create Location(Bill To Location)
2. Update the Contract for Bill to
3. Update Payment Stream(BillToChange)
4. Update Units Billto (Attach Units)
5. Update Bulk Units Billto

Why this matters: This confirms that bulk-unit applications are not only flagged differently; they also have an extra child-record update step specifically for the bulk-units dataset.

Result: When a bulk-unit application uses a different billing address, TeamDesk creates the bill-to location in Aspire, updates the contract, refreshes the payment stream, and synchronizes both attached units and bulk-unit child records to that new bill-to location.

Mid Term Upgrades (MTU)

Mid Term Upgrade processing is implemented as a separate Credit Application path with its own send button, existing-contract lookup, placeholder creation, and dedicated document-submission buttons.

1. Send MTU Application

Description: This button starts the Mid Term Upgrade process for a Credit Application marked as an MTU.

Why this step exists: An MTU does not create a brand-new contract from scratch. Instead, it starts from an existing Aspire transaction and prepares the record for an upgrade flow.

When it is available: The button is shown when:

- Credit Application Type = Midterm Upgrade
- Midterm Upgrade Flag is not checked

What TeamDesk does: The button first sets Credit Application Status to `"Submitted"`, then runs:

1. Get by Trans# (Mid Term Upgrade)
2. Get Account Information(Mid Term Upgrade)
3. Get Location Info(Mid Term Upgrade)
4. Add Account Number
5. Create Placeholder For Mid Term Upgrade
6. Update MTU Field
7. Update the First Asset Trans#
8. MTU Email Alert to PWP Accounting

Result: The MTU application is initialized from the existing Aspire transaction and prepared for upgrade-specific asset and document handling.

---

2. Add Upgrade Documents

Description: This button is used to add MTU documents when there is no DB key.

Why this step exists: If there is no DB key linking the MTU back to Opportunity, TeamDesk cannot fetch the document automatically.

When it is available: The button is shown when:

- Number of Units to be Upgraded = MTU Units Submitted
- Document Status is blank
- DBKEY# is blank
- Midterm Upgrade Flag = true
- override / status conditions are met

What TeamDesk does: This button sets Midterm Upgrade Flag to `True` and then runs Document Navigate to route the user into the manual document-upload path.

Result: The user is routed into the manual MTU document-submission workflow.

---

3. Attach MTU Document

Description: This button is used when an MTU already has a DBKEY# and TeamDesk can pull the document from Opportunity.

Why this step exists: If the MTU is already linked to Opportunity through the DB key, TeamDesk can retrieve the upgrade packet directly.

When it is available: The button is shown when:

- Number of Units to be Upgraded = MTU Units Submitted
- Document Status is blank
- Midterm Upgrade Flag = true
- DBKEY# is populated
- ContractStatus is not `"Dealer Submitted"`
- override conditions are met

What TeamDesk does: This button runs:

1. Get Opportunity Information(Data Only, MTU)
2. Get Opportunity MTU Document
3. Update the Opportunity Flag(MTU)
4. Remove the xml Doc

Result: The MTU document package is retrieved from Opportunity and document-tracking values are updated.

---

4. Attach MTU Document.

Description: This is a second DB-key-based MTU document button used under a slightly different eligibility rule set.

Why this step exists: This path appears to support an alternate approved MTU document-retrieval scenario based on CreditDecision and user access.

When it is available: The button is shown when:

- Number of Units to be Upgraded = MTU Units Submitted
- Document Status is blank
- Midterm Upgrade Flag = `True`
- Credit Application Type = `"Midterm Upgrade"`
- DBKEY# is populated
- CreditDecision contains `"Approved"`
- user Goods&Service Access = `True`
- override conditions are met

What TeamDesk does: This button runs:

1. Get Opportunity Information(Data Only, MTU)
2. Get Opportunity MTU Document
3. Remove the xml Doc
4. Update the Opportunity Flag(MTU)

Result: Approved MTU records with a DB key can retrieve their upgrade packet from Opportunity under this alternate approval/access path.